

#UNIT-3

Why Testing?

- 1) To point out the defects and errors that were made during the development phase
- 2) ~~Make~~ Makes sure of the customer's reliability and their satisfaction in the app
- 3) To ensure quality of the product
- 4) For effective performance of product
- 5) To ensure that the app should not result into any failures
- 6) To stay in the business

####

When to start testing?

- 1) Early testing preferred
- 2) In SDLC, testing can be started from req. gathering phase & continued till the deployment phase
- 3) Also depends on the development model

When to stop testing?

Based on following aspects:

- (i) Testing deadlines
- (ii) Completion of functional & code coverage
- (iii) Management decision
- (iv) Bug rate falls below a certain level

Who does testing?

- 1) Software tester
- 2) Project lead/manager
- 3) Software developer
- 4) End user

Q Verification vs validation

< Already done in Unit 1 >

Testing objectives -

- 1) Testing shows the presence of bugs
- 2) Exhaustive testing is impossible
- 3) Early testing
- 4) Defect clustering
- 5) Pesticide paradox
- 6) Testing is context dependent
- 7) Absence of errors fallacy

Test information flow -



Software Testing Life Cycle (STLC) -

Phases:

- 1) Requirement Analysis
- 2) Testing Planning
- 3) Test case development
- 4) Environment setup
- 5) Test execution
- 6) Test cycle closure

~~R = E = R.~~

RTM = Requirement Traceability Matrix

Summary Table -

Phase	Key activities	Deliverables
1) Requirement Analysis	(i) Identify test case types, priorities, RTM, environment, (ii) Automation feasibility	(i) RTM (ii) Feasibility report
2) Test planning	(i) Create test strategies (ii) Select tools (iii) Estimate effort (iv) Plan resources	(i) Test plan (ii) Effort estimation
3) Test case development	(i) Write test cases/scripts (ii) Review (iii) Prepare test data	(i) Test cases (ii) Scripts (iii) Test data
4) Test environment setup	(i) Setup environment (ii) Prepare data (iii) Smoke testing	(i) Ready environment (ii) Smoke results
5) Test execution	(i) Run tests (ii) Log defects (iii) Map to RTM (iv) Retest (v) Track defects	(i) Updated test cases (ii) Defect reports (iii) RTM status
6) Test cycle closure	(i) Evaluate coverage (ii) Metrics (iii) Map to RTM reporting (iv) Re test defect (v) Track def analysis	(i) Test closure report (ii) Test metrics

Test Cases -

- (i) Defined as inputs + pre conditions + expected results + objective
- (ii) Used to verify system behavior vs requirements
- (iii) Why?
 - Ensure requirement validation
 - Cover specific execution paths
 - Help identify design/requirement defects

Test Case Management Tools -

- (i) Used to manage, track, and maintain test cases
- (ii) Key features:
 - Test case documentation
 - Execution & result tracking
 - Defect tracking automation
 - Traceability
 - Test case protection & versioning
- (iii) eg - JIRA, Quality Center

Black box Testing -

- (i) Tests inputs → outputs with out internal code knowledge

- (ii) Focus on functionality, not implementation
- (iii) Validate system behavior against requirements

- (iv) Key goals:

- Reduce no. of test cases while maintaining coverage
- Detect classes of errors, not just specific cases

- (v) Techniques:

- Equivalence Partitioning (divide inputs into valid/invalid groups → test one from each)
- Boundary Value Analysis (test values at edges)
- Cause-Effect Graphing (map input conditions → output effects → derive test cases)



1) Equivalence Partitioning -

- (i) Divide inputs into equivalence classes (valid & invalid)
- (ii) Test one representative value per class
- (iii) Types

- Valid class
- Invalid class

(iv) Guidelines

- Range input $\rightarrow$ 1 valid + 2 invalid classes
- Specific value $\rightarrow$ 1 valid + 2 invalid
- Set membership $P \rightarrow$ 1 valid + 1 invalid
- Boolean $\rightarrow$ 1 valid + 1 invalid

(v) eg (6-10 valid)

- Invalid: < 6
- Valid: 6-10
- Invalid: > 10
- Sample tests: 4, 7, 12
 X, $\checkmark$, X

2) Boundary Value Analysis -

- (i) Focus on edge values (boundaries)
- (ii) Errors ~~can~~ occur mostly at min/max limits
- (iii) Test:

- Min, Max
- Just inside / just outside

(iv) eg (8-12 valid)

Test cases:

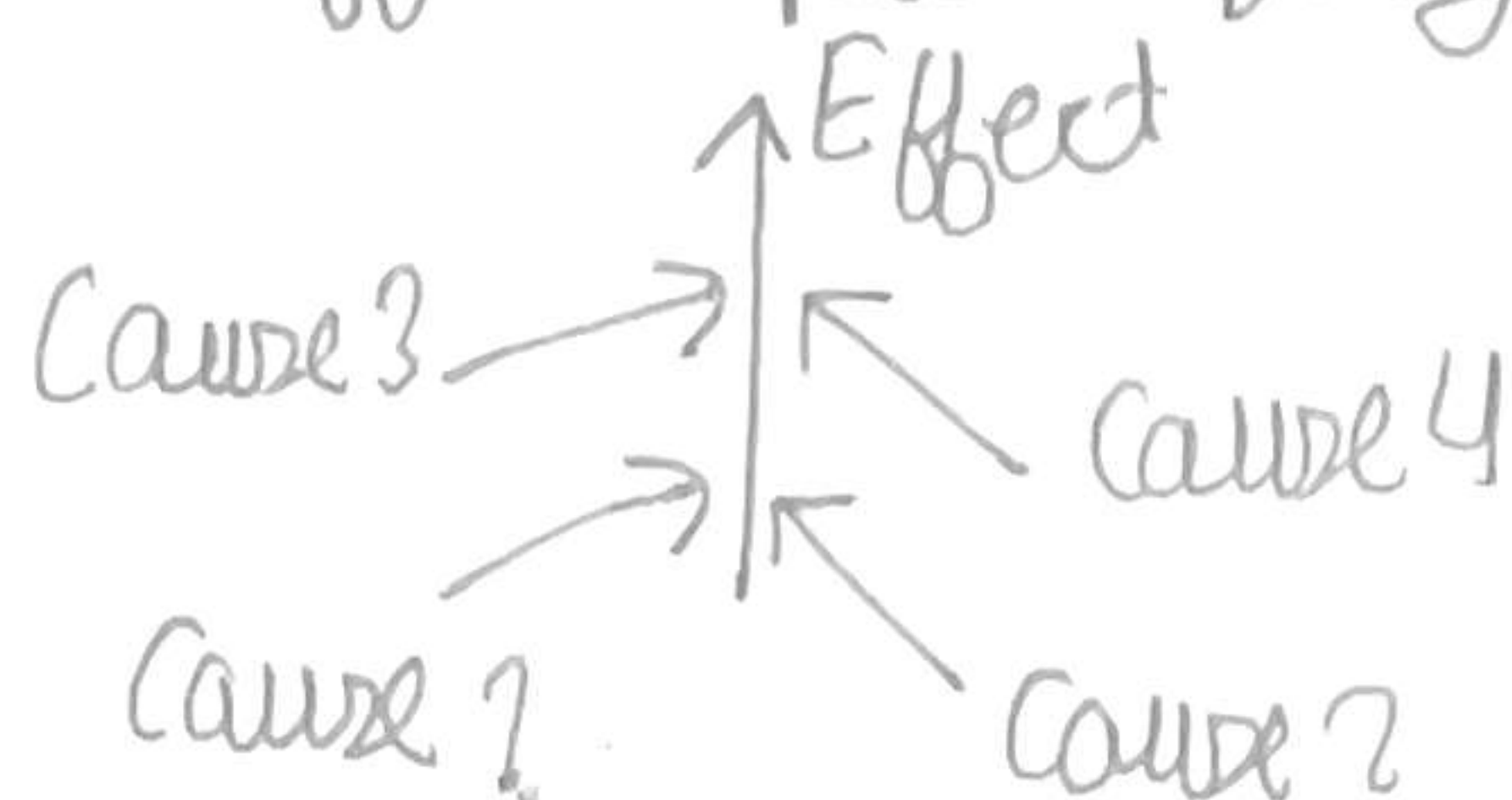
- < 8 (invalid)
- 8 (boundary)
- 9-11 (valid)
- 12 (boundary)
- > 12 (invalid)

3) Cause-Effect Graphing -

- (i) Maps inputs (causes) $\rightarrow$ outputs (effects)
- (ii) Used to derive logical test cases
- (iii) Covers logical combinations of inputs & outputs efficiently
- (iv) Steps:

- Identify causes (inputs) & effects (outputs)
- Build cause-effect graph
- Convert to decision table
- Generate test cases from rules

Cause Effect - Flow Diagram



Cause = input condition
 Effect = output / result
 Maps inputs → outputs logically
 Used to derive test cases systematically

eg- Input: 2 characters

1st → A or B

2nd → Digit

Valid case → File updated

Invalid cases:

Wrong 1st char → Message X

Wrong 2nd char → Message Y

TC1: A + digit → File updated

TC2: B + digit → File updated

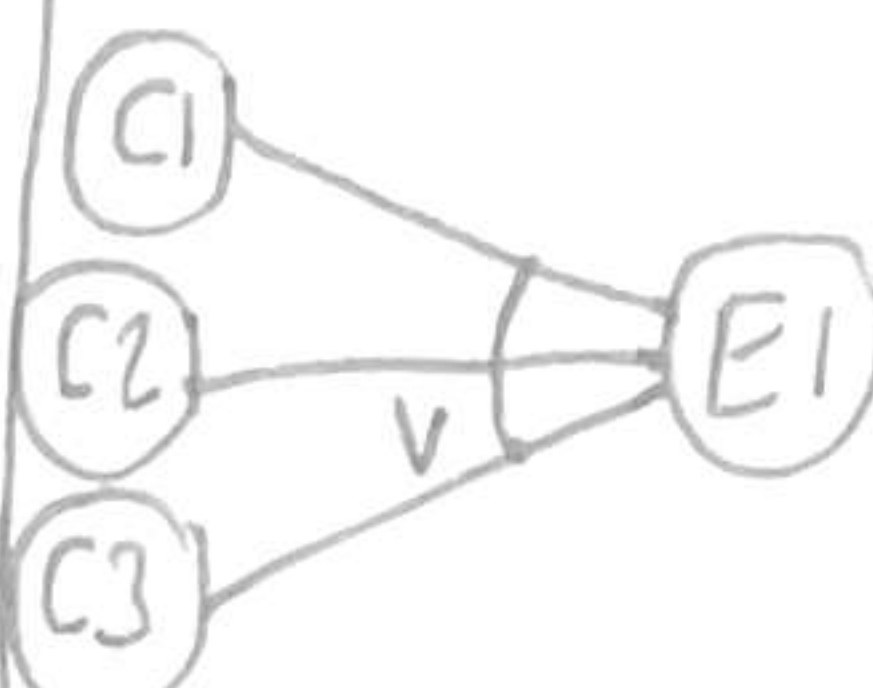
Notations



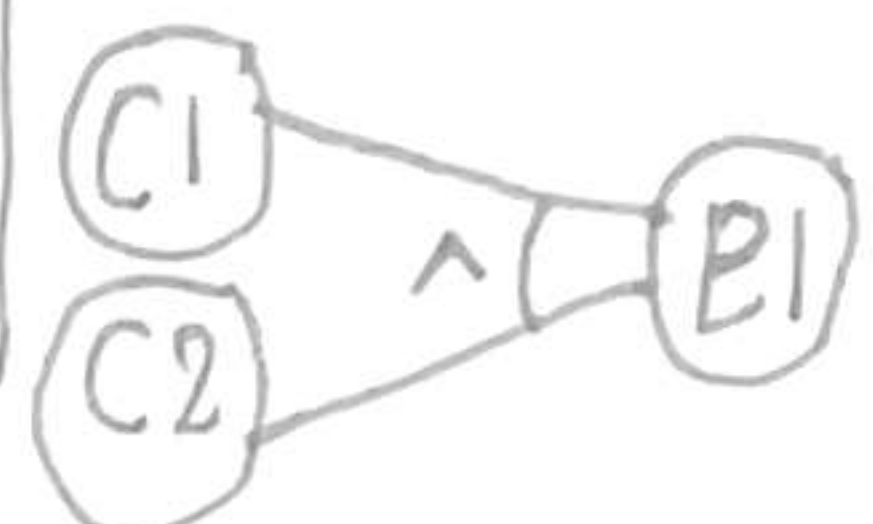
Meaning
Identify



NOT



OR



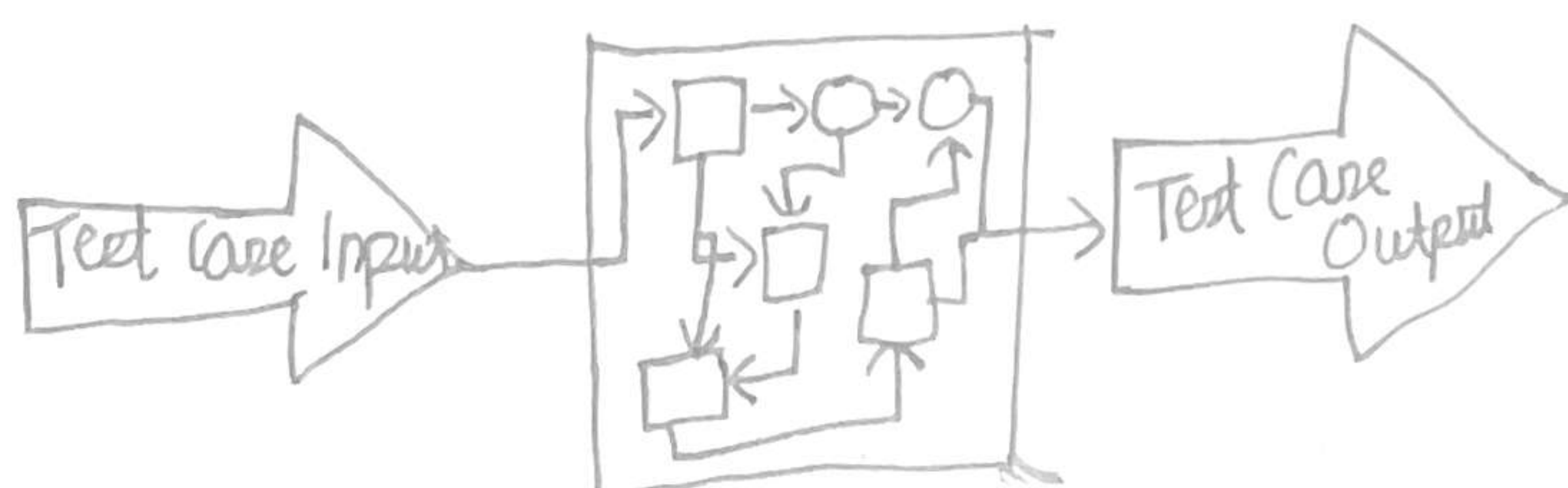
AND

White box Testing -

(i) Tests internal structure / code logic

(ii) Also called: Clear / Glass / Open box testing

(iii) Focus: paths, conditions, loops, data structure



(iv) Purpose / Usage

└ Detect logical errors

└ Used for debugging & code validation

└ Applied at:

└ Unit testing

└ Integration & system levels

(V) Techniques:

(a) Statement Coverage

- Execute all statements at least once
- Formula:
$$\rightarrow (\text{Executed} / \text{Total Statements}) \times 100$$
- Limitation: missed false conditions

(b) Branch (Decision) Coverage

- Test both true & false outcomes
- Formula:

$$\rightarrow (\text{Executed decisions} / \text{Total decisions}) \times 100$$

(c) Path Coverage

- Test all possible execution paths
- Ensured full logic coverage

(vi) eg - Coverage Insight

(a) Statement coverage:

- Needs at least one test case

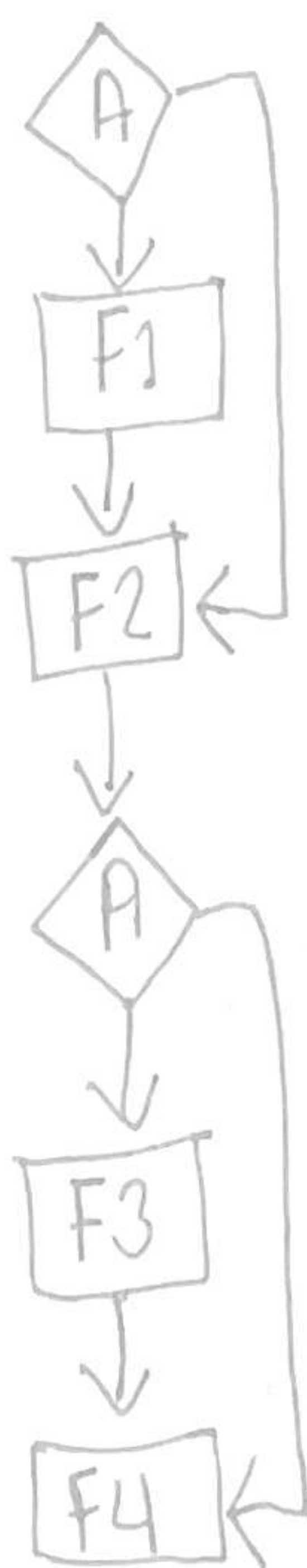
(b) Decision coverage:

- Needs both true & false cases

(c) eg test set:

- TC1: $X > Y$
- TC2: $X < Y$

eg - if (A)
F1();
F2();
if (A)
F3();
F4();



Paths:

A-F1-F2-A-F3-F4
A-F2-A-F3-F4
A-F1-F2-A-F4
A-F2-A-F4

Problem: only 2 are feasible

A=T

A=F

Blackbox vs Whitebox Testing -

Aspect	Blackbox	Whitebox
1) Focus	Inputs & outputs (functionality)	Internal code & logic
2) Knowledge needed	No code knowledge	Requires programming knowledge
3) Performed by	Testers	Developers/Testers
4) Level	Higher level (System, acceptance)	Lower level (Unit, Integration)
5) Type	Functional/external testing	Structural/internal testing
6) Goal	Check what system does	Check how system works
7) Basis	Requirements/specification	Design/code structure
8) Coverage	Limited to functionality	Covers paths, branches, loops
9) Techniques	EP, BVA, Cause-effect	Statement, Branch, Path coverage
10) Start Point	Requirement docs	Design/code docs

Software maintenance -

(i) Process of modifying a software design system after delivering in order to correct faults.

(ii) Activities:

- Fixing bugs & defects
- Adding new features
- Updating software
- Improving performance & reliability

(iii) Importance:

- Keep software usable on newer platforms
- Correct errors discovered during use
- Enhance functionality as user requirements change
- Improve system performance & security
- Reduce long term costs

(iv) Objectives:

- Correct faults
- Maintain security
- Adapt software
- Improve quality
- Enhance features

(v) Types:

- Corrective
- Adaptive
- Perfective
- Preventive

- ### 1) Corrective Maintenance
- Fix bugs/errors after deployment
 - Ensured system works as intended
 - Key activities
 - Detect & fix errors
 - Root cause analysis
 - Apply patches
 - Testing fixes
 - Deploy updates

- ### 2) Adaptive Maintenance
- Modify software for environment change
 - Ensures compatibility
 - Key activities:
 - Analyze environment changes
 - Update code/configurations
 - Validate compatibility
 - Deploy modifications
 - Update documentation

- ### 3) Perfective Maintenance
- Improve performance, usability, features
 - Based on user feedback & evolving needs
 - Key activities
 - Analyze enhancement requests
 - Improve performance/features
 - Refactor code
 - Add upgrades
 - Test improvements

1) Preventive Maintenance -

- Prevent future issues
- Improves reliability & stability
- Key activities
 - Code reviews & analysis
 - Apply updates / security
 - Optimize resource usage
 - System audits & health checks
 - Backup & disaster recovery

* Summary -

- Corrective → Fix errors
- Adaptive → Adjust to changes
- Perfective → Improve system
- Preventive → Avoid future problems

Phases of Software Maintenance -

- 1) Request analysis
- 2) Problem identification
- 3) Solution design
- 4) Implementation of changes
- 5) Testing
- 6) Deployment

Software Maintenance Process Models -

- 1) ~~Process~~ Quick fix model
- 2) Boehm's model
- 3) Reuse oriented model
- 4) Iterative enhancement model
- 5) Tape maintenance model

Software Maintenance Process -

- | | |
|---------------------------|--|
| 1) Problem identification | 6) Deployment |
| 2) Impact analysis | 7) Deployment Documentation updates |
| 3) Design implementations | |
| 4) Implementation | |
| 5) Testing | |

- ## # Challenges -
- High cost & effort
 - Poor documentation
 - Legacy systems
 - Technical debt
 - Growing complexity

- ## # Best Practices to date documentation
- Maintain up to date documentation
 - Use version control to manage changes & enable rollback
 - Automated testing ensures new changes
 - Monitor performance
 - Regular code reviews

Software Re-engineering -

- (i) Process to analyze, modify, transform existing software
- (ii) Improves quality, maintainability, performance
- (iii) Avoids full redevelopment
- (iv) Why?

- High failure rate / poor design
- Outdated or missing documentation
- Obsolete technology
- Changing business requirements

(v) Process

- Planning
- Analysis
- Design
- Implementation
- Testing
- Deployment

(vi) Key Techniques

- Inventory Analysis → Identify candidates
- Document reconstruction → update docs
- Reverse engineering → extract design from code
- Code reconstruction → improve code structure
- Data ~~refa~~ restructuring → modernize data formats
- Forward engineering → rebuild improved system

(vii) Cost Factors

- Software size & complexity
- Documentation quality
- Tools / technologies used
- Team expertise
- Amount of changes needed

(viii) Advantages

- Reduce cost & risk
- Improved maintainability
- Retains business logic
- Better adaptability
- Enhanced quality

(ix) Challenges

- Resource intensive
- Resistance to change
- May disrupt operations
- Hard to measure success

Software Project Management -

- (i) Applies management principles to software projects
- (ii) Activities: planning, organizing, monitoring, controlling
- (iii) Ensures quality deliver within time & budget
- (iv) Handles complexity, uncertainty, changing requirements
- (v) Objectives:

- Deliver on time & budget
- Reduce risks
- Ensure quality
- Optimize resource
- Improve communication

(vi) Characteristics of software projects:

- Intangible (hard to visualize progress)
- Complex & people intensive
- Frequent req. changes
- Risk prone & uncertain

(vii) Role of Project Manager

- Plan & execute project
- Manage scope, time, cost, quality
- Allocate resources & wage
- Communicate with stakeholders
- Handle risks

(viii) Software Project Planning

- Define scope & objectives
- Identify deliverables & milestones
- Estimate cost, effort, duration
- Prepare schedule & resources
- Provide execution roadmap

(ix) Project Estimation Techniques

- Expert judgement
- Analogy with past projects
- Function Point Analysis
- COCOMO model
- Supports budgeting

(x) Project Scheduling

- Break project into tasks
- Defines sequence & dependencies
- Assigns timelines
- Tools: Gantt Chart, CPM
- Identifies critical paths & deadlines

(xi) Risk management

- Identify potential risks
- Analyze impact & probability
- Prioritize risks
- Create mitigation & contingency plans
- Continuous monitoring & control

(xii) Quality management

- Ensures software standards & quality
- Includes planning, assurance, control
- Uses review, inspection, testing
- Focus on defect prevention

(xiii) Configuration management

- Controls changes in software artifacts
- Manages versions & baselines
- Tracks change requests
- Ensures consistency & traceability
- Supports controlled software evolution

(xiv) Project Monitoring & Control

- Tracks progress vs plan
- Measures schedule & cost variance
- Detects deviations early
- Takes corrective actions
- Ensures project stays on track

(xv) Importance of Software Project Management - *

- Improves success rate
- Reduces cost & schedule overruns
- Enhances software quality
- Ensures customer satisfaction
- Provides structured & controlled development

CASE Tools -

- (i) CASE = Computer Aided Software Engineering
- (ii) Software tools that support SDLC activities
- (iii) Improve productivity & software quality
- (iv) Used for analysis, design, coding, testing, documentation
- (v) Objectives

- ~~Need~~ Automate software development activities
- Improve software quality & consistency
- Reduce development time & cost
- Enhance documentation accuracy
- Support better project management

(vi) Need

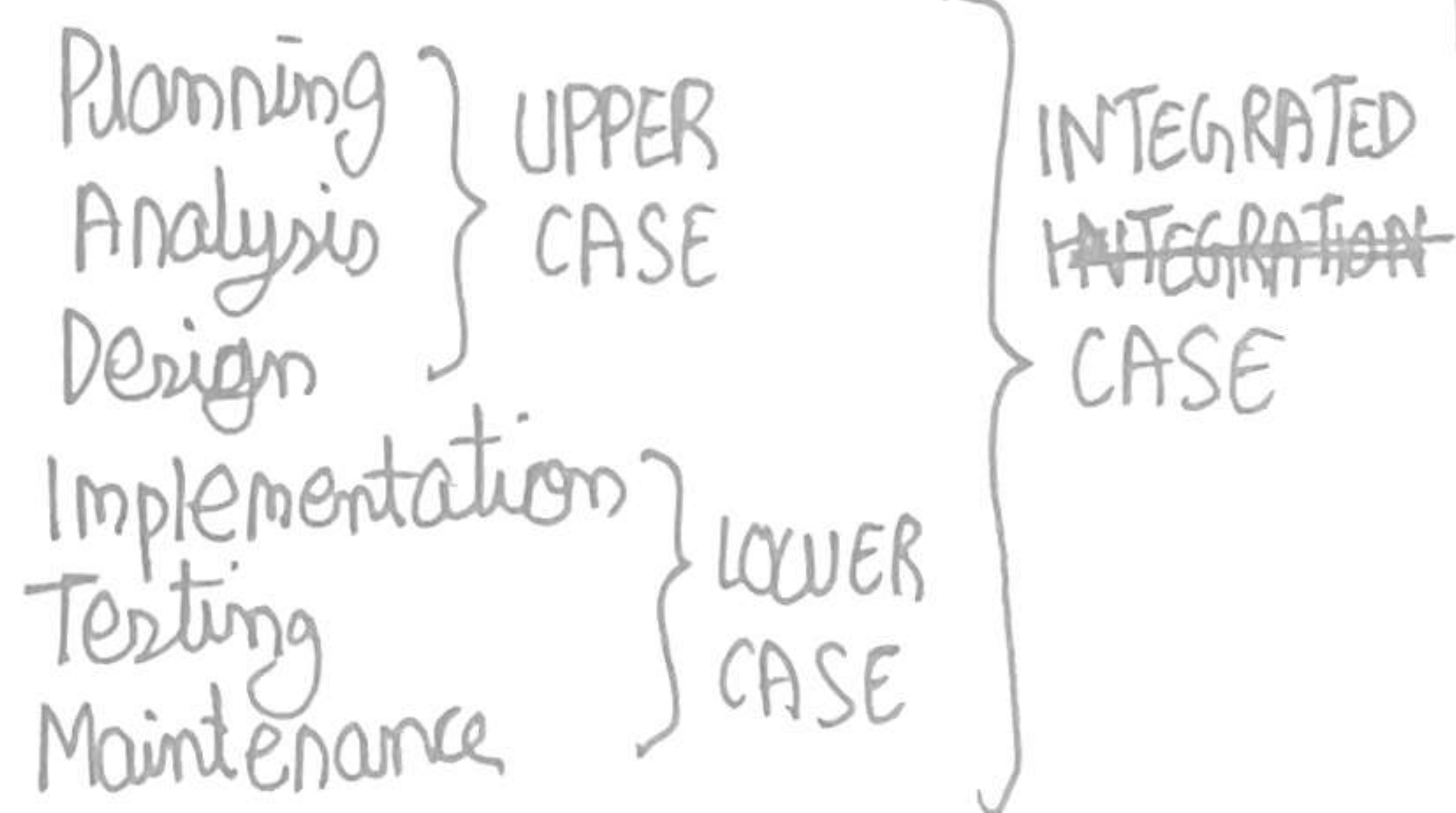
- Software projects are large & complex
- Manual development is error prone
- Frequent requirement changes
- Need for standardization & control
- Easier maintenance & evolution

(vii) Components

- Tools for specific activities
- Central ~~tendency~~ repository for project data
- Standards & methodologies
- User interface for interaction

(viii) Types

- Upper CASE tools
- Lower CASE tools
- Integrated CASE tools



(ix) Advantages

- ~~Increased~~ Increased productivity
- Improved software quality
- Reduced development cost
- Better documentation
- Easier maintenance

(x) Limitations

- High initial cost
- Requires training
- Not suitable for all projects
- Tool integration challenges

(xi) Role

- Support reverse engineering
- Help understand legacy systems
- Manage changes efficiently
- Improve maintainability

Upper CASE tools (Planning & Analysis) -

- (i) Used in early SDLC phases
- (ii) Focus: requirements + system design
- (iii) Functions:
 - └ Requirements management → gather & track requirements
 - └ Diagramming / modeling → system architecture, data flow
- (iv) eg
 - └ DOORS, Jama, Accept 360
 - └ Visio, Enterprise Architect, Lucidchart

Lower CASE tools (Development & Implementation) -

- (i) Used in coding & development phase
- (ii) Functions
 - └ IDEs → code writing & debugging
 - └ Code generators → design → code
- (iii) eg
 - └ VScode, Eclipse, IntelliJ
 - └ IBM Rational Rose

Verification & Validation Tools -

- (i) Ensures software quality before deployment
- (ii) Functions
 - └ Automated testing
 - └ Debugging
- (iii) eg
 - └ Selenium, JUnit, qTest
 - └ GDB, Visual Studio Debugger

Maintenance & Support Tools -

- (i) Used post deployment
- (ii) Functions
 - └ Bug tracking
 - └ Configuration management (SCM)
- (iii) eg -
 - └ Bugzilla, ServiceNow
 - └ Git, Github, SVN

#Cross Life Cycle Tools -

(i) Used across multiple SDLC phases

(ii) Functions

- Project management → planning, scheduling
- Documentation generalization

(iii) eg

- Jira, Trello
- Doxygen, Sphinx

#Configuration Management -

(i) Controls changes in software artifacts

(ii) Manages versions & baselines

(iii) Tracks change requests

(iv) Ensures consistency & traceability

(v) Supports controlled evolution

Software Metrics -

(i) Quantitative technique measure for software projects, processes,

Projects
(ii) Provide objective data for planning

(iii) Improve quality

(iv) Objectives

- Measure size, complexity, effort, cost, quality
- Support estimation & planning
- Monitor progress
- Improve process efficiency
- Aid managerial decisions

(v) Characteristics

- Quantitative (numeric)
- Objective (consistent, unbiased)
- Language independent
- Economical (cost < benefit)

(vi) Classification

- Product metrics (software characteristics)
- Process metrics (development process)
- Project metrics (project management)
- Internal metrics (code level)
- External metrics (user level)

(vii) Benefits

- Better resource allocation
- Early error detection
- Improved product quality

(viii) Challenges

- Lack of standard definition
- High implementation cost
- Risk of metric misuse

Process Metrics -

(a) Improve development & maintenance process

(b) DRE (Defect Removal Efficiency)

↳ defects removed before efficiency

(c) Cycle Time → time to complete task

Product Metrics -

(a) LOC (Lines of Code) → size measure

(b) Cyclomatic complexity → no. of independent paths

(c) Function Points (FP) → functionality measure

Project Metrics -

(a) Monitor progress & performance

(b) Cost variance = $\frac{\text{actual cost} - \text{planned cost}}{\text{cost}}$

(c) Schedule Variance = $\frac{\text{actual time} - \text{planned time}}{\text{time}}$

(d) Track staffing / productivity trends